

Scenario 1: High-Volume Retail Application

Context:

You are designing the backend architecture for **ShopBurst**, an e-commerce platform that targets flash sales events like Black Friday and Cyber Monday. On regular days, the system handles around **50,000 transactions per day**, but during flash sales, this spikes to **500,000 transactions per hour**. The platform has users in North America, Europe, and Asia, so it must handle traffic across multiple time zones.

Here's what the stakeholders want:

- **Marketing Team:** Real-time inventory updates across all regions, so customers see accurate stock counts. They also want promotional notifications delivered instantly.
- **Operations Team:** Warns that real-time synchronization will require highly distributed infrastructure, which could increase system latency during peak loads.
- **Customer Experience Team:** Says users expect seamless interactions—no lag in loading product pages or processing orders.
- **Finance Team:** Wants the architecture to avoid excessive infrastructure costs, especially for servers that are underutilized outside of sales events.

To complicate matters:

1. Inventory for flash sales is limited and needs to be tracked accurately, or customers may abandon the platform due to overselling.
2. The CEO has mandated that **"a laggy experience is unacceptable"** because competitors are quick to capitalize on negative feedback.

Challenge for Architects:

1. What tradeoffs do you foresee in designing a system to meet these requirements?
2. How would you design the architecture to handle inventory tracking, latency, and scalability without breaking the budget?
3. What specific strategies would you recommend (e.g., caching, eventual consistency, database sharding), and how would they affect the system's behavior under peak load?

-
- Scalability vs. Consistency
 - Performance vs. Maintainability
 - Security vs. Usability
 - Cost vs. Quality

- Time to Market vs. Feature Completeness
- Flexibility vs. Complexity
- Fault Tolerance vs. Performance
- Data Storage vs. Processing Speed
- Decoupling vs. Coordination Complexity
- Simplicity vs. Feature Richness